

Statement of purpose

Tiny Driver

Tiny Driver is a pseudo-3D driving game inspired by Atari's Night Driver implemented on a single Tiny Tapeout GF180nm tile. You steer a car down an endless winding road between pylons, trying to get as far as possible within a time limit.

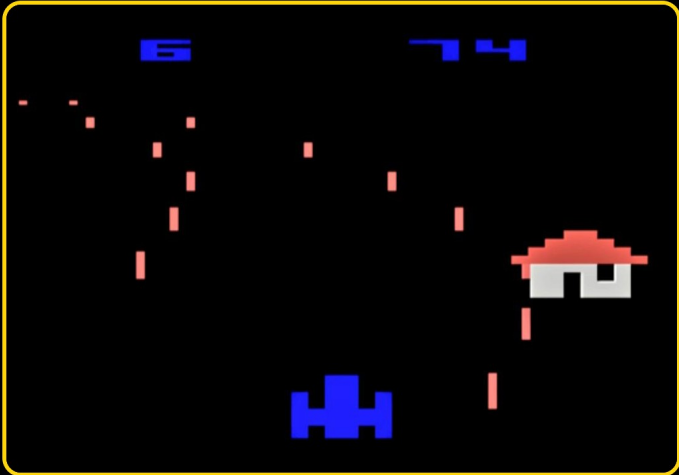
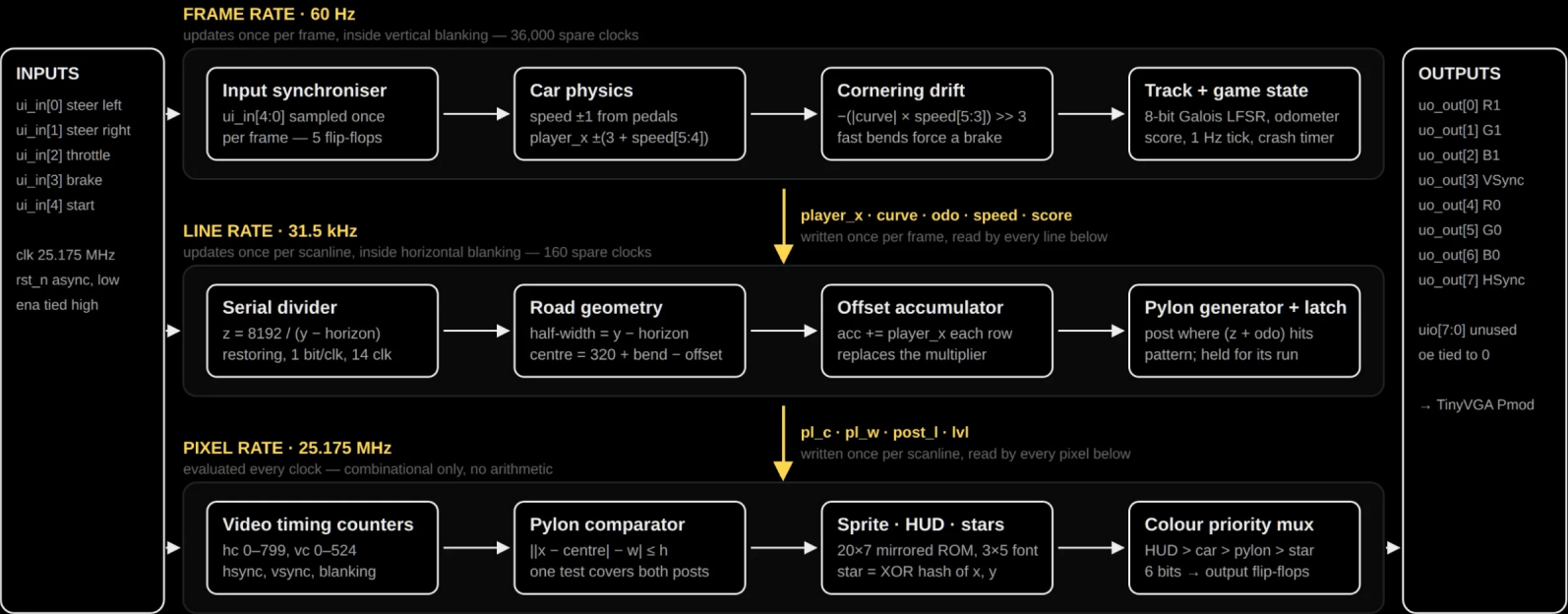
How to play

Steer left or right, throttle, brake. One point per pylon pair passed, against a countdown; clip a pylon and you lose time.

Proposed specification

Process	GF180MCU (gf180mcuD)	Die area	1 tile, (160 μm × 100 μm)
Clock	25.175 MHz, one domain	Reset	rst_n, async, active low
Video	640 × 480 @ 60 Hz VGA	Colour	6 bits, 2 per channel
Standard cells	under 1,800	Flip-flops	about 175

System diagram



Night Rider, Atari (1980)

Visual Breakdown

The car is a fixed sprite at the center of the screen. Steering slides the road one way, the winding road slides it the other; playing the game is cancelling the two to remain centered. Let either win and the nearest post reaches the center, you have crashed.

How each frame is generated

- No framebuffer or memory
- Graphics are generated per pixel (score, car, pylons, star, etc.)

Depth creation (Psuedo-3D)

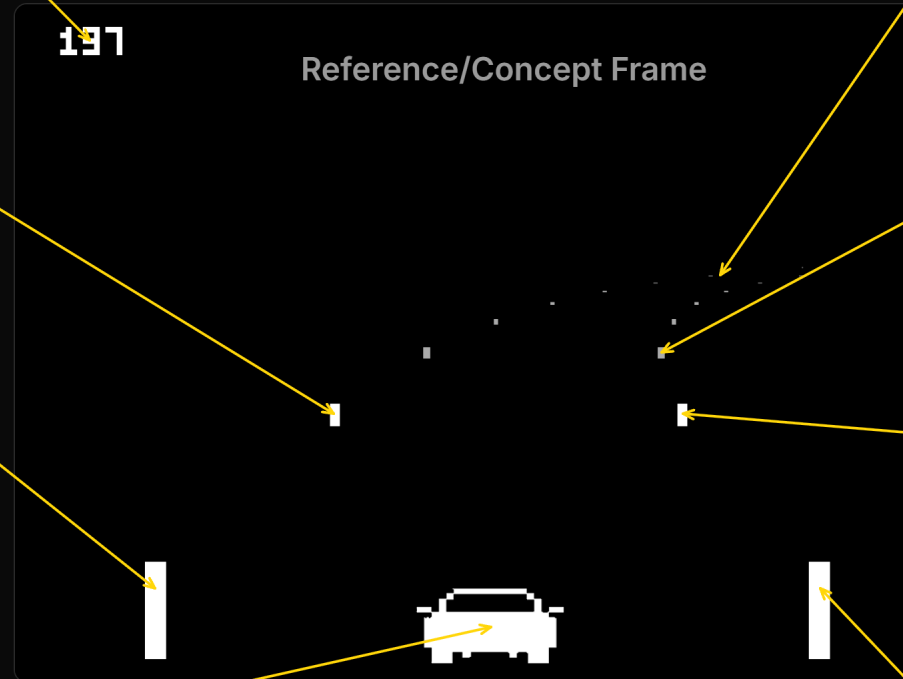
- Every pixel on a row is equally far
- 525 divides a frame
- One bit per clock, between lines

Pylon size and brightness (Psuedo-3D)

- Road width is the depth number
- Post height scales with it
- Creates a horized perspective

Fixed car sprite

- Fixed at bottom centre, every frame
- Single 20×7 sprite
- The "road" slides past it instead



Turning

- Far rows shift more than near rows
- That is a curve, in perspective
- One add per row, no geometry

Movement animation

- No list of pylons is stored anywhere
- A post exists on a row where depth + odometer hits the pattern
- Odometer grows → next frame the same post lands one row lower
- Speed = how much you add per frame

Crashing

- One compare per frame, on the most front pylon
- Further out than the road is wide?
- If pylon reaches center = crashed

I/O pin assignment

Pin	Dir	Signal	Function
ui_in[0]	in	STEER_L	Steer left
ui_in[1]	in	STEER_R	Steer right
ui_in[2]	in	THROTTLE	Accelerate
ui_in[3]	in	BRAKE	Brake
ui_in[4]	in	START	Start / restart
ui_in[7:5]	in	—	Unused
uo_out[0]	out	R1	Red, high bit
uo_out[1]	out	G1	Green, high bit
uo_out[2]	out	B1	Blue, high bit
uo_out[3]	out	VSYNC	Frame sync, low
uo_out[4]	out	R0	Red, low bit
uo_out[5]	out	G0	Green, low bit
uo_out[6]	out	B0	Blue, low bit
uo_out[7]	out	HSYNC	Line sync, low
uio[7:0]	—	—	Unused, oe = 0
clk	in	CLK	25.175 MHz
rst_n	in	RST_N	Async reset, low
ena	in	ENA	High from harness

Timeline

Oct 15	Video timing, road and pylon rendering correct in simulation; first cocotb tests passing.
Oct 22	Game logic complete: physics, drift, crash, score. First full hardening to measure tile utilization / area.
Nov 5	Feature freeze & Gate-level simulation against the same cocotb suite as the RTL.
Nov 15	Post-layout verification
Nov 20	Final GDS submitted with info.yaml and documentation.

Task Breakdown

Matthew Embaye

Design

Architecture & RTL, graphic gen., divider and geometry, game logic and sprites

Yacine Ouhrouche

Verification

Testing, cocotb suite, simulation, area and timing review

Both: Documentation & final submission.

Risk plan: Area is the largest constraint, so it is measured on Oct 22 with potential feature changes to meet spec. The twelve blocks are independently testable, so partial progress is always demonstrable.